

# Design Review: whattheflip

**Module:** INF2004 Embedded Systems Programming  
**Group ID:** SE06 (Lab P1)  
**Project Title:** whattheflip — Embedded Multi-Bus Sniffer, Wireless Monitor & Chaos Fuzzer  
**Target Hardware:** ESP32-C6 RISC-V @ 160 MHz + Raspberry Pi Pico  
**Operating System:**  $\mu\text{T-Kernel}$  3.0 ( esp32c6-mtk3 )  
**Submission Milestone:** Week 6 Architecture Freeze

## Purpose

Document the system requirements, hardware connections, RTOS task structure, behavioural models, assumptions, and AI-supported design reasoning before major implementation. Treat this Week 6 submission as an architecture freeze before full integration and Tribunal 1. The design may change later, but major changes must be justified with engineering evidence.

## Team Role and Subsystem Ownership

| Buddy   | Student Name | Student ID    | Primary Subsystem / Responsibility                                                                                                                                                                                                                     |
|---------|--------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buddy 1 | Tan Yan Qi   | Student Entry | <b>Wired Protocol Auto-Discovery Engine:</b> 4-pin signal timing acquisition, rise-time measurement, UART/I2C/SPI classification, and dynamic pin role mapping.                                                                                        |
| Buddy 2 | Annie Goh    | Student Entry | <b>Physical Chaos &amp; Fault Injection Engine:</b> Frame-synchronized glitch injection (bit-flips, I2C ACK suppression, clock stretching), bus contention protection, and hardware E-stop driver release.                                             |
| Buddy 3 | Zayne Lee    | Student Entry | <b>Wireless Multi-Protocol Sniffer Engine:</b> ESP32-C6 2.4 GHz promiscuous radio capture for Wi-Fi 6 (802.11ax) management frames and IEEE 802.15.4 (Zigbee 3.0), extracting RSSI, channel, and timestamps.                                           |
| Buddy 4 | Ryan Ke      | Student Entry | <b>Telemetry Ingestion &amp; PCAP Storage Engine:</b> Zero-copy fixed memory ring buffers, packet serialization into standard libpcap format (.pcap), and high-speed FAT32 MicroSD card logging.                                                       |
| Buddy 5 | Joseph Poon  | Student Entry | <b>System Supervisor, Web UI &amp; Victim Control:</b> $\mu\text{T-Kernel}$ supervisor coordination, WebSocket live streaming dashboard over local Wi-Fi, hardware E-stop interrupt handling, NeoPixel/buzzer indicators, and victim stimulus control. |

## 1. Requirements

### 1.1 Functional Requirements (FR)

| **FR1** | The system shall passively acquire digital signals across 4 probe pins with an input leakage current  $\leq 1\,\mu\text{A}$  ( $I_{in} \geq 1\,\text{M}\Omega$ ) and a timing resolution  $\leq 1\,\mu\text{s}$  (backed by

\$\mu\text{T-Kernel}\$ 1 MHz physical timer `tk_ptmr`). || **FR2** | The auto-discovery engine shall classify standard UART (9600\text{ to }115200\text{ baud} \pm 2\%), standard I2C (100\text{ kHz and }400\text{ kHz}), and SPI Mode 0 (up to 1\text{ MHz}) signals within \$\le 500\text{ ms}\$ of continuous bus activity. || **FR3** | The system shall programmatically map any of the 4 probe pins to internal protocol decoding channels without requiring physical jumper wire alterations. || **FR4** | The wireless engine shall capture raw 802.11 management frames (beacons, probe requests) and raw IEEE 802.15.4 frames across 2.4 GHz channels 11–26, extracting received signal strength (RSSI), channel ID, and microsecond timestamps. || **FR5** | When armed by the user, the chaos engine shall inject non-destructive protocol faults into the target bus, specifically single bit-inversions, I2C ACK \$\to\$ NACK suppressions, and clock stretching up to 50\mu\text{s} \pm 10\%. || **FR6** | The system shall transmit targeted over-the-air stress frames (including 802.11 deauthentication frames) to trigger reconnection routines on the target wireless node. || **FR7** | Actuation of the physical emergency stop button shall disarm all active output drivers and return all probe lines to high-impedance (High-Z) mode within \$\le 5\text{ ms}\$. || **FR8** | The storage engine shall write captured packet records containing microsecond timestamps and raw packet bytes to non-volatile removable media in standard `libpcap` ( `.pcap` ) binary format. || **FR9** | The system shall broadcast decoded packet summaries and bus error statistics over a local wireless interface to connected client dashboards at a periodic refresh rate of 1\text{ Hz to }2\text{ Hz} (\$500\text{ ms to }1000\text{ ms}\$), with event-driven immediate alerts for fault states. |

1.2 Non-Functional Requirements (NFR)

| ID   | Requirement Statement                                                                                                                                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR1 | The emergency stop and safety disarm routines shall preempt all background processing tasks within \$\le 5\text{ ms}\$ of physical switch contact.                                                                                |
| NFR2 | The firmware shall perform zero dynamic heap allocations ( <code>malloc/free</code> ) during active monitoring and fault injection phases.                                                                                        |
| NFR3 | The capture pipeline shall achieve 0\% packet drop rate for continuous wired streams up to 500\text{ kbps} (covering UART 115200 and I2C Fast Mode), and buffer wireless bursts of up to 32 consecutive frames without data loss. |
| NFR4 | The physical probe interface shall tolerate continuous input voltages up to 5.5\text{ V} without exceeding the microcontroller pad maximum rating of 3.6\text{ V}.                                                                |
| NFR5 | Fault injection pulses shall synchronize to target bus trigger edges with a timing jitter of \$< 500\text{ ns}\$ (well within the 8.68\mu\text{s} bit window of 115200 baud UART).                                                |

2. Hardware Integration Pin Layout

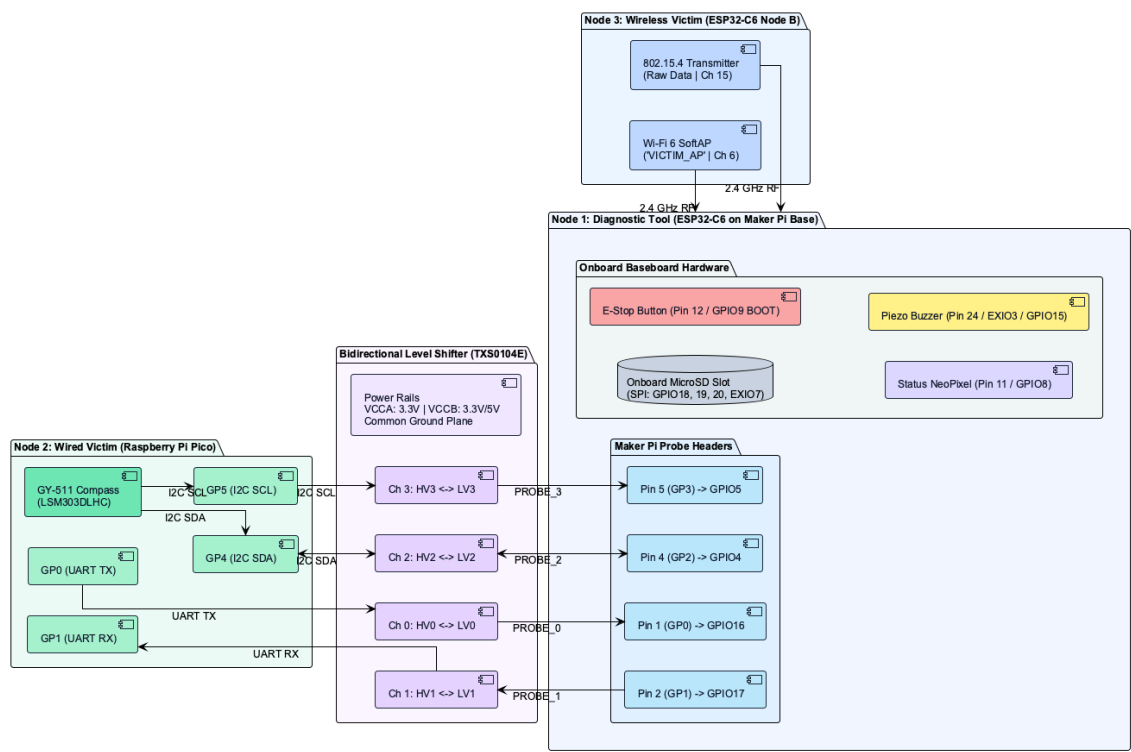
2.1 Hardware Interconnect & Maker Pi Pico Base Integration

The diagnostic tool (Node 1) consists of the **Waveshare ESP32-C6-Pico-M** mounted directly onto the **Cytron Maker Pi Pico Base** carrier board. This integration provides three major hardware advantages:

- 1. **Integrated MicroSD Card Socket:** Utilizes the onboard spring-push MicroSD slot hardwired to physical pins 14–16, eliminating loose external SPI breakout modules.
- 2. **Per-Pin Real-Time LED Diagnostics:** Every GPIO header pin on the Maker Pi Base features an onboard blue LED indicator, providing immediate physical visual feedback when probe lines and SPI buses toggle.

3. **Electrical Safety Invariant (I2C Bus Protection)**: On the ESP32-C6-Pico, physical pins 26 and 27 connect to native `GPIO22` (SDA) and `GPIO23` (SCL), which drive the onboard TCA9554PWR expander. The team has mandated using the **native BOOT pushbutton on Pin 12 ( GPIO9 )** for Emergency Stop, strictly avoiding Maker Pi Base Buttons 1 and 2 (GP20/GP21) which would otherwise short the expander's I2C bus to ground.

Figure 1: Hardware Interconnect & Maker Pi Base Pinout



2.2 Pin Allocation Table (Maker Pi Base Carrier Integration)

Note: All 4 probe pins are dynamically shared between `AutoDiscovery_Task` (listening mode) and `Chaos_Fuzzer_Task` (injection mode), arbitrated via a mutex ( `MTX_PROBE_ACCESS` ). Mounting the ESP32-C6 on the Maker Pi Base maps physical header pins to native high-speed GPIOs for sniffing, while utilizing the onboard SD card socket and WS2812 LED.

| Maker Pi Pin | Signal Name | ESP32-C6 Pin | Interface Type  | Owning Task / Arbitration       | Electrical & Hardware Notes                                                              |
|--------------|-------------|--------------|-----------------|---------------------------------|------------------------------------------------------------------------------------------|
| Pin 1 (GP0)  | PROBE_0     | GPIO16       | GPIO / Edge IRQ | Shared (AutoDiscovery / Fuzzer) | High-speed native GPIO. Level-shifted to Wired Victim GP0. Baseboard blue LED indicator. |
| Pin 2 (GP1)  | PROBE_1     | GPIO17       | GPIO / Edge IRQ | Shared (AutoDiscovery / Fuzzer) | High-speed native GPIO. Level-shifted to Wired Victim GP1. Baseboard blue LED indicator. |

|                          |            |                           |                            |                                    |                                                                                                                  |
|--------------------------|------------|---------------------------|----------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Pin 4<br/>(GP2)</b>   | PROBE_2    | <b>GPIO4</b>              | GPIO /<br>Edge IRQ         | Shared (AutoDiscovery /<br>Fuzzer) | High-speed native<br>GPIO. Level-shifted<br>to Wired Victim GP4<br>(SDA). Baseboard<br>blue LED indicator.       |
| <b>Pin 5<br/>(GP3)</b>   | PROBE_3    | <b>GPIO5</b>              | GPIO /<br>Edge IRQ         | Shared (AutoDiscovery /<br>Fuzzer) | High-speed native<br>GPIO. Level-shifted<br>to Wired Victim GP5<br>(SCL). Baseboard<br>blue LED indicator.       |
| <b>Pin 14<br/>(GP10)</b> | SD_SCK     | <b>GPIO18</b>             | Hardware<br>SPI            | Storage_Task                       | <b>Hardwired to<br/>Maker Pi onboard<br/>MicroSD slot.</b> 10<br>MHz SPI Clock.                                  |
| <b>Pin 15<br/>(GP11)</b> | SD_MOSI    | <b>GPIO19</b>             | Hardware<br>SPI            | Storage_Task                       | <b>Hardwired to<br/>Maker Pi onboard<br/>MicroSD slot.</b> SPI<br>Master Out Slave In.                           |
| <b>Pin 16<br/>(GP12)</b> | SD_MISO    | <b>GPIO20</b>             | Hardware<br>SPI            | Storage_Task                       | <b>Hardwired to<br/>Maker Pi onboard<br/>MicroSD slot.</b> SPI<br>Master In Slave Out.                           |
| <b>Pin 20<br/>(GP15)</b> | SD_CS      | <b>EXIO7</b>              | IO<br>Expander<br>/ CS     | Storage_Task                       | <b>Hardwired to<br/>Maker Pi onboard<br/>MicroSD slot.</b><br>Active-Low Chip<br>Select via TCA9554.             |
| <b>Pin 12<br/>(GP9)</b>  | E_STOP_BTN | <b>GPIO9</b>              | GPIO<br>Input<br>(Pull-Up) | Safety_Supervisor_Task             | <b>Onboard BOOT<br/>pushbutton.</b> Active-<br>Low, $\leq 5\text{ms}$ High-Z<br>interrupt (avoids I2C<br>short). |
| <b>Pin 11<br/>(GP8)</b>  | STATUS_LED | <b>GPIO8</b>              | WS2812 /<br>DIN            | Safety_Supervisor_Task             | <b>Onboard WS2812<br/>Addressable RGB<br/>LED</b> (Green: Ready,<br>Blue: Sniff, Red:<br>Fault).                 |
| <b>Pin 24<br/>(GP18)</b> | BUZZER_OUT | <b>EXIO3 /<br/>GPIO15</b> | PWM /<br>Audio             | Safety_Supervisor_Task             | Maker Pi Base<br>onboard piezo<br>buzzer (Pin 24) or<br>external resonant<br>buzzer on GPIO15.                   |

|                     |         |                    |                |                       |                                                                                                      |
|---------------------|---------|--------------------|----------------|-----------------------|------------------------------------------------------------------------------------------------------|
| <b>Pins 26 / 27</b> | EXP_I2C | <b>GPIO22 / 23</b> | I2C Master Bus | System Init           | Dedicated to onboard TCA9554PWR expander.<br><b>Baseboard Buttons 1 &amp; 2 must NOT be pressed.</b> |
| <b>Pin 36 (3V3)</b> | VCC_LV  | <b>3V3 OUT</b>     | Power Output   | Hardware Interconnect | 3.3V reference power to low-voltage side of TXS0104E level shifter.                                  |
| <b>Pin 38 (GND)</b> | GND     | <b>GND</b>         | Ground Plane   | Hardware Interconnect | Common reference ground unified across Maker Pi Base, Level Shifter, and Pico Victim.                |

### 3. RTOS Task Table ( $\mu\text{T-Kernel}$ 3.0)

| Task Name              | Purpose                                                                   | Inputs                                      | Outputs                                   | Trigger                           | Priority           |
|------------------------|---------------------------------------------------------------------------|---------------------------------------------|-------------------------------------------|-----------------------------------|--------------------|
| Safety_Supervisor_Task | Monitor emergency stop, system faults, and enforce High-Z probe isolation | GPIO9 E-stop IRQ, watchdog, bus short flags | Force High-Z registers, RGB LED, buzzer   | Event-driven (IRQ / Fault flag)   | <b>Highest (1)</b> |
| Wireless_Engine_Task   | Drain 2.4 GHz radio baseband FIFOs and extract packet metadata            | Radio promiscuous RX baseband event         | Ingest packet records into memory pool    | Event-driven (Radio Baseband IRQ) | <b>High (2)</b>    |
| Chaos_Fuzzer_Task      | Inject synchronized glitch pulses into target bus transmissions           | Trigger pattern config, edge events         | Timed GPIO glitch pulse (bit-flip / NACK) | Event-driven (Frame sync event)   | <b>Medium (3)</b>  |
| AutoDiscovery_Task     | Sample edge transitions, measure pulse widths,                            | Probe edge FIFO timestamps                  | Resolved protocol ID, pin role table      | Periodic (50 ms) / Edge event     | <b>Medium (4)</b>  |

|                    |                                                                       |                                             |                                   |                                 |            |
|--------------------|-----------------------------------------------------------------------|---------------------------------------------|-----------------------------------|---------------------------------|------------|
|                    | and resolve pin mappings                                              |                                             |                                   |                                 |            |
| Storage_Task       | Serialize packet buffers to standard .pcap files on MicroSD card      | Ingested packet records from message buffer | 512-byte FAT32 disk sector writes | Event-driven (Buffer threshold) | Low (5)    |
| Web_Dashboard_Task | Serve HTTP dashboard assets and stream live telemetry over WebSockets | Decoded packet headers, bus error stats     | Asynchronous TCP/IP Wi-Fi frames  | Periodic (500–1000 ms)          | Lowest (6) |

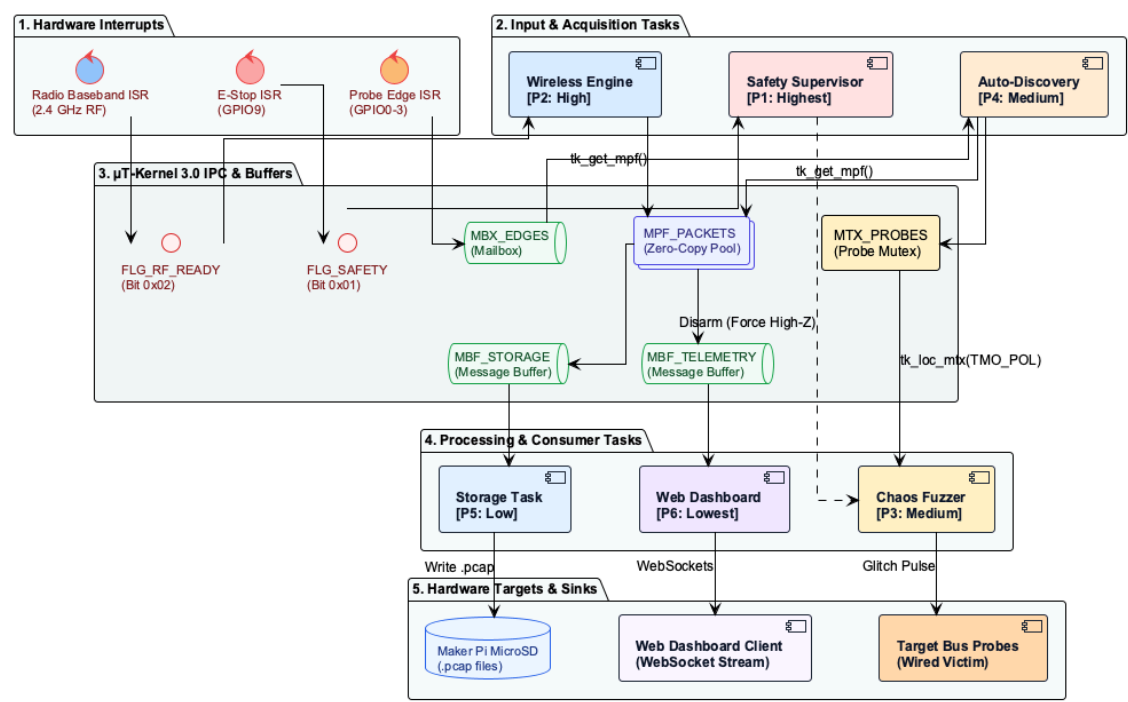
**Priority and Ownership Justification:**

`Safety_Supervisor_Task` is allocated Priority 1 (highest) to guarantee that an emergency stop actuation or electrical contention event preempts all other tasks within  $\leq 5\text{ ms}$ , immediately floating all probe pins to protect connected target hardware. `Wireless_Engine_Task` is set to Priority 2 to prevent hardware radio FIFO overruns during dense 802.11 beacon bursts. `Chaos_Fuzzer_Task` sits at Priority 3 to ensure deterministic, sub-microsecond glitch timing when a target frame trigger is recognized. `AutoDiscovery_Task` sits at Priority 4 to perform edge timing analysis without preempting active frame capture. `Storage_Task` and `Web_Dashboard_Task` are assigned the lowest priorities (5 and 6) because disk and network operations are decoupled through pre-allocated fixed memory pools ( `MPF_PACKETS` ), ensuring slow SD card sector writes never block time-critical bus sampling.

---

**4. Task Interaction Diagram**

Figure 2:  $\mu$ T-Kernel 3.0 Task Interaction & IPC Architecture

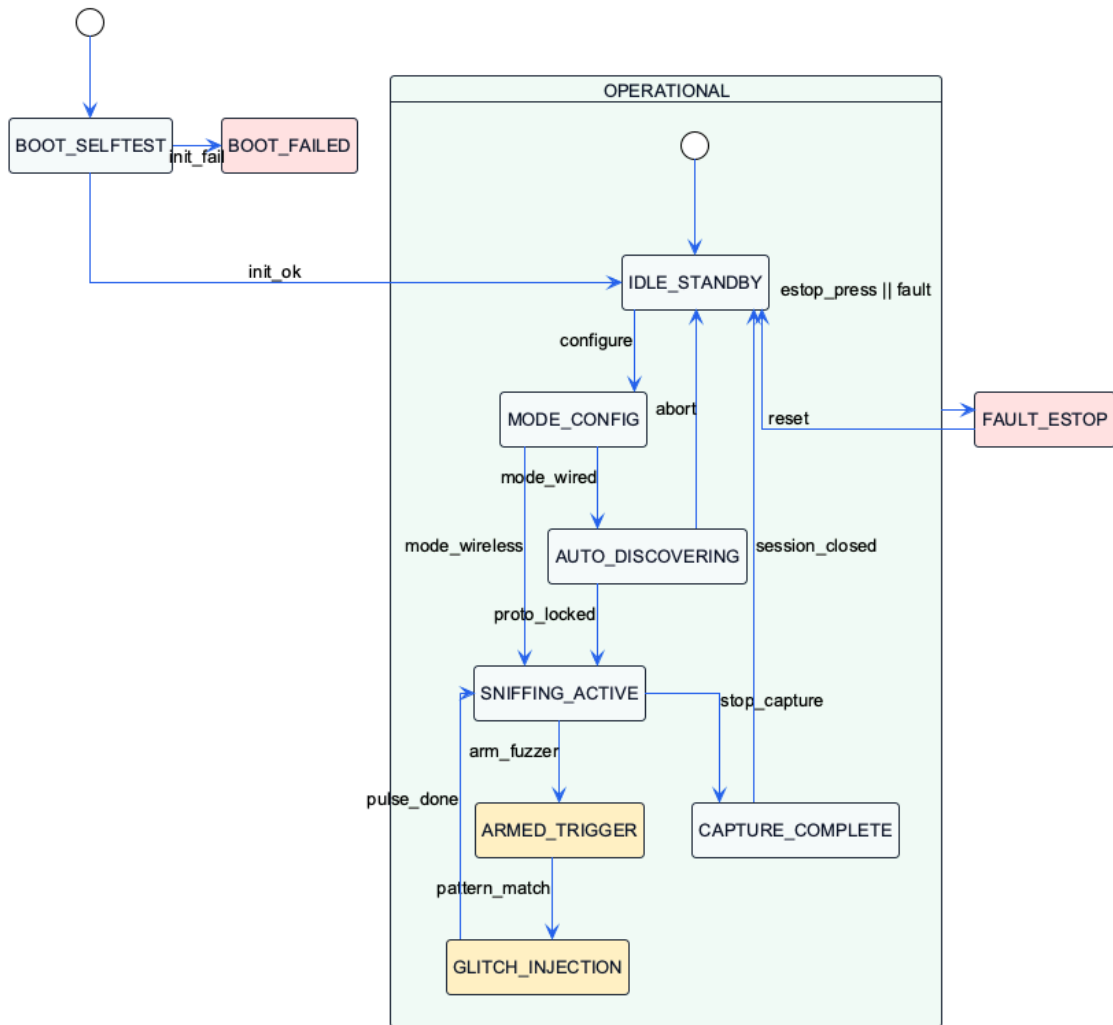


Explanation of Main Communication Paths:

- 1. Safety Interrupt Path:** When `GPIO9` falls LOW, `E-Stop ISR` sets EventFlag `FLG_SAFETY_ABORT`, waking `TSK_SAFETY` (Priority 1) to directly write to GPIO output-enable registers, floating all probe pins to High-Z within  $\leq 5\text{ ms}$ .
- 2. Probe Arbitration & Timing Path:** `Probe Edge ISR` captures hardware timer timestamps and enqueues them via Mailbox `MBX_EDGES` to `TSK_DISCOVERY`. Probe pin access between discovery and fuzzing is serialized via Mutex `MTX_PROBE_ACCESS`.
- 3. Zero-Copy Ingestion & Storage Pipeline:** When `Radio ISR` receives an 802.11 or 802.15.4 frame, it allocates a buffer from Fixed Memory Pool `MPF_PACKETS` without dynamic `malloc`. The populated packet is referenced into Message Buffers `MBF_STORAGE` and `MBF_TELEMETRY`. Even if a MicroSD card write incurs a  $30\text{ ms}$  latency spike, the pre-allocated memory pool absorbs incoming bursts without dropping frames.

5. System-Level Statechart

**Figure 3: Unified System-Level Statechart**



**Main States and Assumptions:**

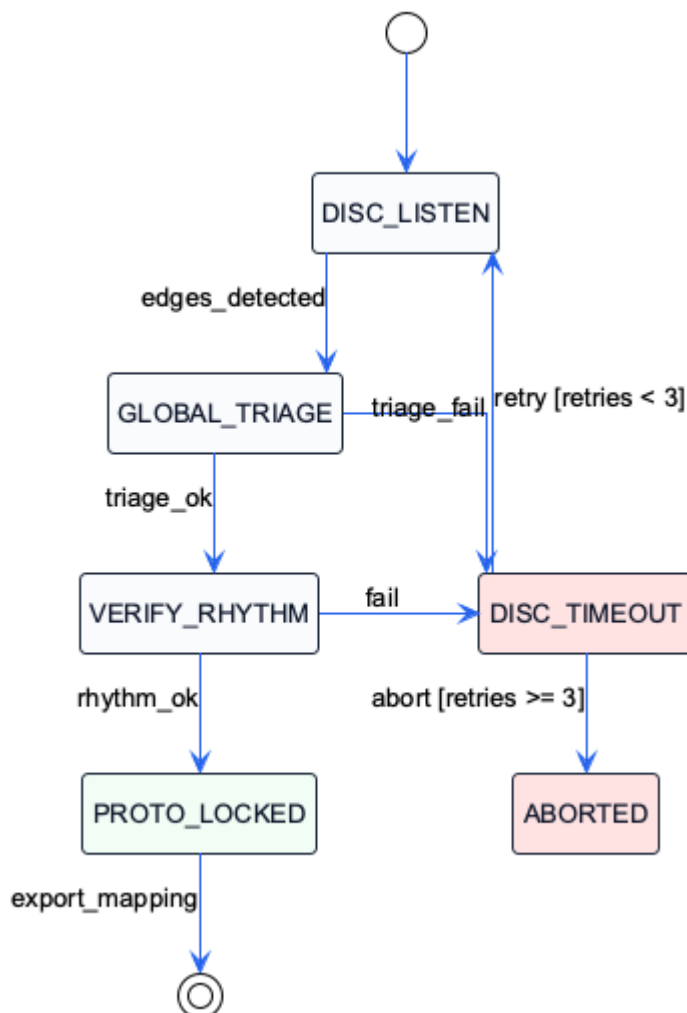
- **BOOT\_SELFTEST** : Verifies MicroSD card mounting, SPI communication, and radio baseband initialization. Transitions to **BOOT\_FAILED** if hardware initialization fails.
- **BOOT\_FAILED** : Terminal fault state halting system execution with a blinking red LED if critical boot hardware is unreadable.
- **IDLE\_STANDBY** : Default safe operational state. Probe pins are configured as High-Z inputs; fuzzer output drivers are completely disabled.
- **MODE\_CONFIG** : Processes user configuration commands to set operating profiles (Wired vs. Wireless, RF channels, or baud search parameters).
- **AUTO\_DISCOVERING** : Measures edge counts, pulse widths, and clock period stability across the 4 probes to resolve protocol type and pinout.
- **SNIFFING\_ACTIVE** : Ingests packet streams into zero-copy memory pools, writing **.pcap** files to MicroSD and broadcasting live WebSocket telemetry.
- **ARMED\_TRIGGER** : Fuzzer armed; synchronizes to target bus framing and awaits a matched address/header trigger pattern.
- **GLITCH\_INJECTION** : Injects a precision in-flight glitch pulse (bit-flip or ACK suppression bounded to  $t < 50, \mu\text{s}$ ) and verifies bus release.

- **CAPTURE\_COMPLETE** : Flushes remaining packet ring buffers to MicroSD and closes active PCAP sessions before returning to **IDLE\_STANDBY** .
- **FAULT\_ESTOP** : Global emergency preemption state invoked by physical E-stop press or bus contention. Forces all probe lines to High-Z in  $\leq 5 \text{ ms}$ , sounds buzzer, and latches until manually reset.

## 6. Subsystem Statecharts

### 6.1 Subsystem Statechart 1: Wired Auto-Discovery Engine

- **Subsystem Name:** Wired Protocol Auto-Discovery Engine
- **Related RTOS Task:** `AutoDiscovery_Task ( TSK_DISCOVERY )`
- **Purpose:** Passively classify unknown bus protocols and resolve signal pin assignments.
- **Reason a Statechart is Needed:** Enforces a multi-stage sequential filter (pin triage  $\rightarrow$  rhythm verification  $\rightarrow$  confirmation) to reject noise spikes and prevent false protocol locks. **Figure 4: Subsystem 1 — Wired Auto-Discovery Statechart**



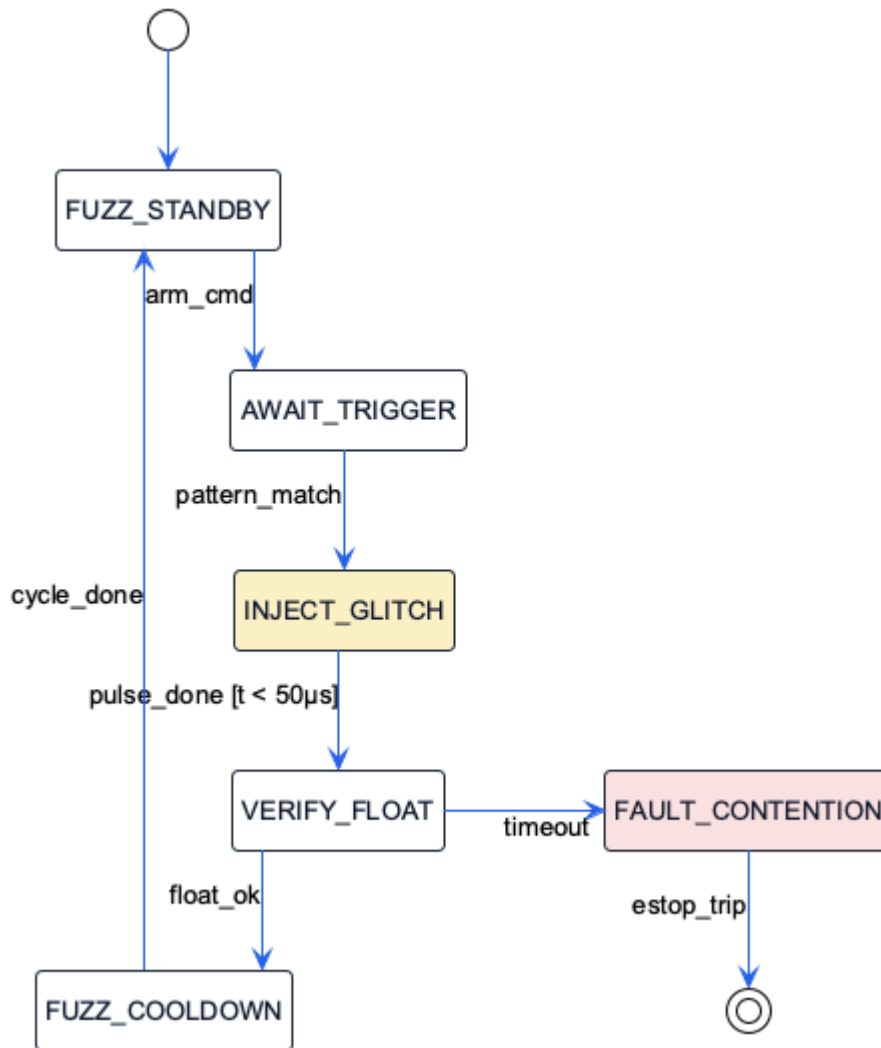
States, Events, and Fault Handling:

- `DISC_LISTEN` : Samples probe transitions. Upon receiving edge interrupts, moves to `GLOBAL_TRIAGE` .
  - `GLOBAL_TRIAGE` : Applies exclusivity rules (1 pin = UART candidate, 2 pins = I2C candidate, 3–4 pins = SPI candidate). If active pin count is invalid, triggers `ev_triage_fail` .
  - `VERIFY_RHYTHM` : Evaluates baud rate pulse widths ( $\pm 5\%$  tolerance) or I2C clock period stability ( $\pm 25\%$  jitter bound).
  - `PROTO_LOCKED` : Emits `ev_proto_locked` , updates shared pinout tables, and releases `MTX_PROBE_ACCESS` .
  - **Fault Handling:** If rhythm verification fails or no valid protocol locks within  $s$ , transitions to `DISC_TIMEOUT` , notifies the dashboard, and retries up to 3 times before resetting to `DISC_LISTEN` .
- 

## 6.2 Subsystem Statechart 2: Chaos Fuzzing Engine

- **Subsystem Name:** Physical Chaos & Fault Injection Engine
- **Related RTOS Task:** `Chaos_Fuzzer_Task` ( `TSK_FUZZER` )
- **Purpose:** Inject synchronized non-destructive faults (bit-flips, ACK suppression, clock stretching) into active bus traffic.
- **Reason a Statechart is Needed:** Guarantees that active electrical drive is strictly bounded in time, preventing bus shorts or damaged target drivers. **Figure 5: Subsystem 2 — Chaos Fuzzing Engine**

## Statechart



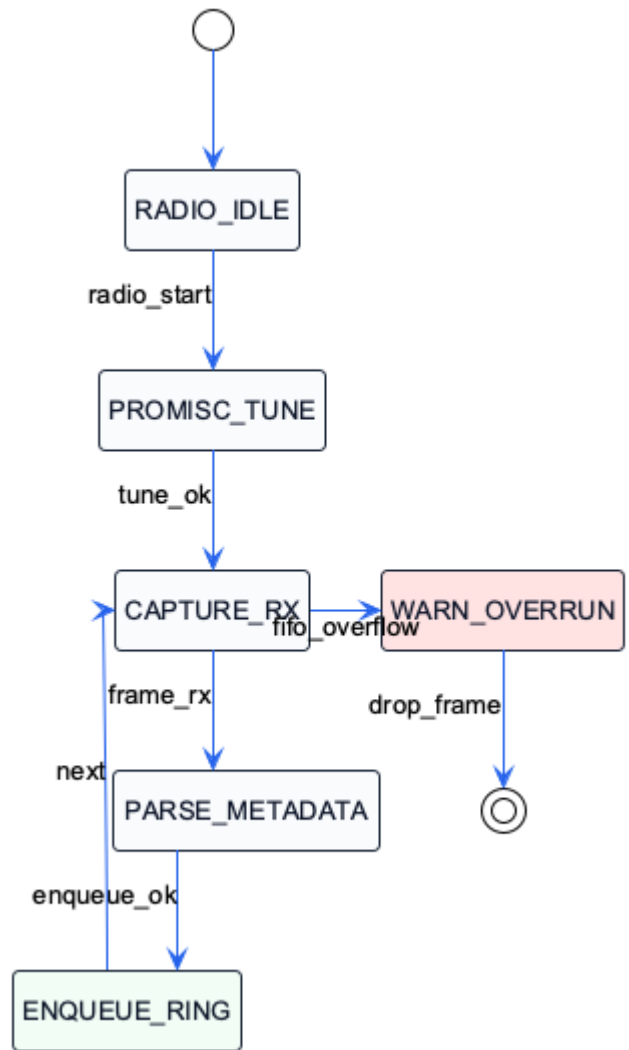
### States, Events, and Fault Handling:

- **FUZZ\_STANDBY** : Probes remain in High-Z mode until user issues arm command.
- **AWAIT\_TRIGGER** : Evaluates incoming bytes to synchronize with the target frame header.
- **INJECT\_GLITCH** : Drives the probe line for a single bit duration (e.g.  $8.68\mu\text{s}$  at 115200 baud).
- **VERIFY\_FLOAT** : Re-samples line state to ensure the line returned to idle HIGH.
- **Fault Handling**: If a line remains driven or pulled LOW for  $> 100\mu\text{s}$ , **FAULT\_CONTENTION** is asserted, immediately switching all outputs to High-Z and alerting **Safety\_Supervisor\_Task**.

## 6.3 Subsystem Statechart 3: Wireless Multi-Protocol Sniffer

- **Subsystem Name**: Wireless Multi-Protocol Sniffer Engine
- **Related RTOS Task**: **Wireless\_Engine\_Task** ( **TSK\_WIRELESS** )
- **Purpose**: Intercept raw 802.11 and 802.15.4 packets and extract RF physical metadata.

- **Reason a Statechart is Needed:** Manages radio baseband transitions and provides graceful degradation during high-throughput packet bursts. **Figure 6: Subsystem 3 — Wireless Multi-**



**Protocol Sniffer Statechart**

**States, Events, and Fault Handling:**

- **RADIO\_IDLE** : Radio disabled; power consumption minimized.
- **PROMISC\_TUNE** : Configures channel filter and engages promiscuous mode.
- **CAPTURE\_RX** : Drains hardware baseband FIFO upon packet arrival.
- **PARSE\_METADATA** : Extracts RSSI, channel ID, and microsecond timestamps into standard PCAP frame format.
- **Fault Handling:** If the fixed memory pool reaches  $> 85\%$  capacity, non-critical broadcast data frames are dropped while management/control frames are preserved, logging an overrun warning to telemetry.

**7. Assumptions**

| ID | Assumption | Risk if Wrong | Validation Method | Owner | Status |
|----|------------|---------------|-------------------|-------|--------|
|----|------------|---------------|-------------------|-------|--------|

|    |                                                                                                                                                            |                                                                                  |                                                                                                                                                                   |             |      |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|------|
| A1 | Probe inputs resolve clean CMOS logic thresholds ( $V_{IL} \leq 0.8V$ , $V_{IH} \geq 2.0V$ ) without excessive ringing on $15\text{ cm}$ jumper lines.     | Edge noise triggers spurious interrupts, causing false baud rate detection.      | Capture $100\text{ kHz}$ I2C transitions on an oscilloscope with $15\text{ cm}$ leads; verify rise time $t_r \leq 300\text{ ns}$ and no false trigger interrupts. | Tan Yan Qi  | Open |
| A2 | ESP32-C6 GPIO pins can switch direction from High-Z Input to Driven Output in $< 1\mu\text{s}$ for targeted bit glitching.                                 | Delayed output drive misses the target clock pulse, failing to inject the fault. | Program a GPIO toggle triggered by an external pulse; measure drive delay on an oscilloscope with $< 50\text{ ns}$ resolution.                                    | Annie Goh   | Open |
| A3 | The ESP32-C6 promiscuous packet filter captures raw 802.11 management frames without dropping frames at burst rates up to $50\text{--}100\text{ pkts/s}$ . | Dropped beacon frames cause incomplete Wireshark trace exports.                  | Blast the sniffer with a $100\text{ pkt/s}$ beacon generator (Node 3); compare transmitted vs. logged frame sequence counters.                                    | Zayne Lee   | Open |
| A4 | SPI FAT32 MicroSD write latency spikes ( $\leq 40\text{ ms}$ ) can be fully absorbed by an $8\text{ KB}$ pre-allocated packet ring buffer.                 | Buffer overflow during SD write stalls drops incoming physical/wireless packets. | Simulate a $50\text{ ms}$ SD write stall while streaming UART traffic at $115200\text{ baud}$ ; verify zero dropped bytes in memory logs.                         | Ryan Ke     | Open |
| A5 | The WebSocket telemetry server can stream $1\text{--}2\text{ Hz}$ packet updates over Wi-Fi without starving the $\mu\text{T-Kernel}$ scheduler.           | Network stack CPU load introduces jitter into physical bus sampling.             | Benchmark system timer jitter while serving $3$ concurrent WebSocket clients running telemetry streams at $1\text{ Hz}$ .                                         | Joseph Poon | Open |

## 8. AI-Supported Design Evidence

### 8.1 Pre-Submission AI Ideation

| Buddy | AI Prompt or Discussion Used | Useful AI Ideas | Weak or Unrealistic AI Ideas | Team Decision |
|-------|------------------------------|-----------------|------------------------------|---------------|
|       |                              |                 |                              |               |

|                |                                                                                                       |                                                                                                                         |                                                                                      |                                                                                                                                      |
|----------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Buddy 1</b> | Prompted AI on multi-pin signal triage heuristics for mixed UART and I2C lines.                       | AI proposed the "Global Pin Exclusivity Filter" (counting active lines before calculating timings).                     | AI suggested using a neural network classifier on the Pico to identify protocols.    | Adopted the deterministic pulse-width heuristic; rejected neural networks as an unnecessary compute and memory burden.               |
| <b>Buddy 2</b> | Asked AI how to safely inject in-flight faults on open-drain I2C buses without frying master drivers. | AI suggested driving lines strictly during slave ACK clock pulses and adding an immediate High-Z release timeout.       | AI suggested cutting the physical VCC line with a MOSFET to simulate device failure. | Implemented software-timed ACK suppression and physical E-stop High-Z override; rejected cutting VCC to prevent back-powering chips. |
| <b>Buddy 3</b> | Inquired whether ESP32-C6 can sniff connected BLE data channels promiscuously.                        | AI identified that ESP-IDF only supports BLE GAP advertisement scanning, not raw data channel sniffing.                 | AI initially claimed BLE promiscuous mode existed via standard APIs.                 | Critiqued and dropped Bluetooth link-layer sniffing; scoped wireless strictly to native Wi-Fi 6 + 802.15.4 promiscuous capture.      |
| <b>Buddy 4</b> | Asked AI how to log high-rate packet streams to MicroSD without missing real-time deadlines.          | AI suggested using pre-allocated fixed memory pools (MPF) and message buffers to decouple capture from SD block writes. | AI suggested writing raw text JSON strings to the SD card for each captured packet.  | Adopted zero-copy fixed memory pools; rejected JSON in favor of compact binary libpcap format.                                       |
| <b>Buddy 5</b> | Asked AI to evaluate whether a 0.96" OLED display should be included alongside the Wi-Fi dashboard.   | AI highlighted that the OLED consumes 1 KB RAM, bus bandwidth, and cannot render complex hex packet trees.              | AI suggested keeping both OLED and Web dashboard for redundancy.                     | Completely dropped the OLED screen; allocated resources to the WebSocket web dashboard and status NeoPixel.                          |

## 8.2 Team-Reviewed Architecture Freeze

| AI or Initial Idea                                                        | Team Decision | Engineering Reason                                                                       | Affected Artefact                                  |
|---------------------------------------------------------------------------|---------------|------------------------------------------------------------------------------------------|----------------------------------------------------|
| Use a 3-node closed-loop testbed (Tool + Wired Victim + Wireless Victim). | Accepted      | Proves technical scope with dedicated, repeatable hardware; eliminates reliance on noisy | Testbed topology, System overview, Hardware pinout |

|                                                                                 |          |                                                                                                                                              |                                                           |
|---------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
|                                                                                 |          | campus Wi-Fi or uncontrolled targets.                                                                                                        |                                                           |
| Mount ESP32-C6 on Cytron Maker Pi Pico Base carrier board.                      | Accepted | Eliminates external MicroSD breakout module by utilizing onboard SD slot; provides per-pin blue LED diagnostics and clean benchtop mounting. | Hardware pinout table, Schematic, Bill of Materials       |
| Drop 0.96" OLED display in favor of Wi-Fi WebSocket dashboard.                  | Accepted | Eliminates I2C bus contention, saves 1 KB RAM, and provides far superior visual clarity during zoom tribunals.                               | Bill of Materials, Pinout table, Task table               |
| Mandate bidirectional 3.3V $\rightarrow$ 5V logic level shifter on probe lines. | Accepted | RP2040 and ESP32-C6 pads have a 3.6V maximum limit; connecting 5V targets directly causes over-voltage destruction.                          | Hardware integration table, Schematic, NFR4               |
| Omit Bluetooth (BLE) promiscuous mode from the project scope.                   | Accepted | Standard ESP-IDF lacks raw link-layer BLE sniffing APIs; initializing NimBLE consumes 45 KB SRAM and 200 KB flash.                           | Protocol matrix, Wireless requirements, Task architecture |

### 8.3 Final AI-Vetted Gap Check

- AI Reviewer Prompt Used:** *"Act as a strict embedded systems reviewer for whattheflip. Check requirements, pinout, RTOS task table, IPC architecture, statecharts, assumptions, and consistency. Identify missing requirements, unclear interfaces, untested assumptions, timing risks, task conflicts, and shared-resource problems."*

| AI-Identified Gap                                                                                         | Team Response           | Action Taken                                                                                                                     | Evidence or Reason                                             |
|-----------------------------------------------------------------------------------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Maker Pi Base buttons 1 & 2 (GP20/21) short the TCA9554 expander I2C bus to GND.                          | <b>Fixed</b>            | Mandated using native BOOT switch on Pin 12 (GPI09) for Emergency Stop, strictly banning baseboard buttons 1 & 2.                | Hardware integration table Section 2.2 and Assumption A2.      |
| Unclear probe line ownership between Auto-Discovery and Fuzzing tasks.                                    | <b>Fixed</b>            | Defined dynamic probe sharing arbitrated via Mutex <code>MTX_PROBE_ACCESS</code> .                                               | Task table Section 3 and IPC diagram Section 4 updated.        |
| MicroSD card write latency spikes ( $\leq 40\text{ ms}$ ) may cause buffer overrun during burst captures. | <b>Needs Validation</b> | Sized fixed memory pool ( <code>MPF_PACKETS</code> ) to 8 KB to buffer up to 32 consecutive frames; validated via Assumption A4. | Assumption A4 documented with simulated 50 ms stall benchmark. |

|                                                                               |                         |                                                                                                              |                                                     |
|-------------------------------------------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Single 2.4 GHz RF front-end cannot capture Wi-Fi and 802.15.4 simultaneously. | <b>Accepted as Risk</b> | Documented time-multiplexed capture constraint; system operates in distinct Wi-Fi or Zigbee listening modes. | Documented in Section 2 and Subsystem Statechart 3. |
|-------------------------------------------------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|

## 9. Design Consistency

- **Which task owns each hardware device?**
  - `AutoDiscovery_Task` and `Chaos_Fuzzer_Task` dynamically share the 4 target probe lines on Maker Pi Pins 1, 2, 4, 5 (ESP32-C6 `GPI016` , `GPI017` , `GPI04` , `GPI05` ), arbitrated via `Mutex MTX_PROBE_ACCESS` .
  - `Wireless_Engine_Task` owns the 2.4 GHz radio transceiver baseband.
  - `Storage_Task` owns the Maker Pi onboard MicroSD card slot hardwired to physical pins 14–16 and 20 (ESP32-C6 `GPI018` , `GPI019` , `GPI020` , and `EXI07` ).
  - `Safety_Supervisor_Task` owns the emergency stop button on Maker Pi Pin 12 ( `GPI09` BOOT switch), status NeoPixel ( `GPI08` ), and alert buzzer ( `EXI03` / `GPI015` ).
  - `Web_Dashboard_Task` owns the Wi-Fi AP network socket interface.
- **Which task has the highest priority, and why?**
  - `Safety_Supervisor_Task` has the highest priority (Priority 1) because target hardware safety is the primary system invariant. If an electrical short occurs or the user presses the emergency stop, the system must immediately preempt all other tasks and drop probe pins to High-Z in  $5\text{ ms}$ .
- **Which subsystem statechart corresponds to which RTOS task or subsystem?**
  - Subsystem Statechart 1 corresponds to `AutoDiscovery_Task` ( `TSK_DISCOVERY` ).
  - Subsystem Statechart 2 corresponds to `Chaos_Fuzzer_Task` ( `TSK_FUZZER` ).
  - Subsystem Statechart 3 corresponds to `Wireless_Engine_Task` ( `TSK_WIRELESS` ).
- **How are functional requirements reflected in the task and statechart design?**
  - FR1–FR3 (Probing, Auto-Discovery, Pin Routing) map to `AutoDiscovery_Task` and the `AUTO_DISCOVERY` state.
  - FR4 (Wireless Capture) maps to `Wireless_Engine_Task` and the `PASSIVE_SNIFF` state.
  - FR5–FR6 (Chaos Fault Injection) map to `Chaos_Fuzzer_Task` and the `CHAOS_FUZZING` state.
  - FR7 (Emergency Stop) maps to `Safety_Supervisor_Task` and the `FAULT_ESTOP` state.
  - FR8–FR9 (PCAP Storage and Web Streaming) map to `Storage_Task` and `Web_Dashboard_Task` .
- **What changes are still expected after the Week 6 review?**
  - Confirmation of empirical rise-time tolerances during Week 8 hardware wiring tests.
  - Fine-tuning of SPI DMA block transfer sizes and ring buffer depths based on measured SD write latency benchmarks.
  - Validation of assumptions A1 through A5 using physical oscilloscope and network traffic generator measurements before Week 10 Subsystems Check-in.

## 10. Appendix: Lecture 6.1 Traceability Matrix (V-Model Verification)

Formatted strictly to the 5-column standard from Lecture 6.1 Slide #5:

Requirement ID → Use Case / Sequence ID → LLD Function Name → Test ID → Verification Status

| Requirement ID            | Use Case / Sequence ID         | LLD Function Name                     | Test ID | Verification Status           |
|---------------------------|--------------------------------|---------------------------------------|---------|-------------------------------|
| FR1 (Passive Probing)     | UC-1.1 (Sample Bus Lines)      | probe_sample_edges()                  | TC-1.1  | Planned (Architecture Freeze) |
| FR2 (Auto-Discovery)      | UC-1.2 (Classify Protocol)     | analyze_for_i2c(), analyze_for_uart() | TC-1.2  | Planned (Architecture Freeze) |
| FR3 (Dynamic Reconfig)    | UC-1.3 (Map Peripheral Pins)   | gpio_matrix_route_pins()              | TC-1.1  | Planned (Architecture Freeze) |
| FR4 (Wireless Capture)    | UC-2.1 (Capture 802.11 Frames) | wifi_promiscuous_rx_cb()              | TC-3.1  | Planned (Architecture Freeze) |
| FR4 (Zigbee Capture)      | UC-2.2 (Capture 802.15.4)      | ieee802154_rx_cb()                    | TC-3.2  | Planned (Architecture Freeze) |
| FR5 (I2C Fault Injection) | UC-3.1 (Suppress I2C ACK)      | fuzzer_inject_i2c_nack()              | TC-2.1  | Planned (Architecture Freeze) |
| FR5 (UART Bit Flip)       | UC-3.2 (Invert UART Bit)       | fuzzer_invert_uart_bit()              | TC-2.2  | Planned (Architecture Freeze) |
| FR6 (Wireless Stress)     | UC-3.3 (Inject Deauth Frame)   | wifi_inject_deauth_frame()            | TC-3.1  | Planned (Architecture Freeze) |
| FR7 (Emergency Stop)      | UC-4.1 (E-Stop Abort)          | safety_abort_to_high_z()              | TC-4.1  | Planned (Architecture Freeze) |
| FR8 (PCAP SD Logging)     | UC-5.1 (Write PCAP Record)     | pcap_write_frame()                    | TC-4.2  | Planned (Architecture Freeze) |
| FR9 (WebSocket Stream)    | UC-5.2 (Stream Telemetry)      | ws_broadcast_telemetry()              | TC-5.1  | Planned (Architecture Freeze) |
| NFR1 (E-Stop ≤ 5ms)       | UC-4.1 (E-Stop Latency)        | safety_abort_to_high_z()              | TC-4.1  | Planned (Architecture Freeze) |

|                                         |                              |                        |               |                               |
|-----------------------------------------|------------------------------|------------------------|---------------|-------------------------------|
| <b>NFR2</b> (Zero Dynamic Heap)         | UC-1.1 (Memory Allocation)   | tk_get_mpf()           | <b>TC-4.2</b> | Planned (Architecture Freeze) |
| <b>NFR3</b> (Burst Reliability)         | UC-2.1 (Queue Reliability)   | ring_buffer_push()     | <b>TC-3.1</b> | Planned (Architecture Freeze) |
| <b>NFR4</b> (Voltage Clamp)             | UC-1.1 (Electrical Clamping) | Hardware Level Shifter | <b>TC-1.1</b> | Planned (Architecture Freeze) |
| <b>NFR5</b> (Jitter $< 500\text{ ns}$ ) | UC-3.1 (Glitch Precision)    | timer_sync_glitch()    | <b>TC-2.1</b> | Planned (Architecture Freeze) |